Neural Pre-Decoding Under Device Calibrated Noise: IBM Calibration Data, Correlated Matching and Integer Inference on One Alveo U55C

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Neural pre-decoders remove local errors from a surface code syndrome before a matching decoder handles the rest, and are usually trained and judged on uniform circuit noise. We evaluate NVIDIA's Fast pre-decoder on memory circuits whose every fault location carries the error rate of a real qubit or coupler, read from IBM calibration CSV files. Eight IBM devices, three placements of code qubits and three error scales give 108 points at distances 3 and 5, with 43,200,000 shots. Against plain matching the pre-decoder wins at 44 of 54 distance 5 points, with failure ratios down to 0.64. Against PyMatching with correlated matching, which needs no network, the outcome reverses: the pre-decoder loses at every distance 5 point (ratios 1.07 to 1.61), and placing it in front of correlated matching does not help. It keeps an advantage at distance 3 and low noise, where correlated matching is worse than plain matching. On strongly uneven qubits the network makes even plain matching worse, by up to 1.17 times, while matching weights from each qubit's calibration beat device medians by up to 1.16 times. The integer network runs on one Alveo U55C card, bit exact in all 18 runs and, in the 14 runs with a CPU counterpart, identical to the CPU integer model in logical failures shot by shot. One card decodes 2,330 distance 5 windows per second, which a 256 thread CPU exceeds by 2.4 times. Calibration files, code and a teaching example for distances 1 to 5 are released.

Keywords: Quantum error correction, Surface code, Neural pre-decoder, Device calibration, Correlated matching, FPGA, Integer inference, Alveo U55C

1 Introduction

A surface code logical qubit is protected only as long as its syndrome is decoded fast and well [1, 2]. Experiments that operate below the surface code threshold [3, 4] have made decoding speed and accuracy questions of system design, and minimum weight perfect matching, in the Sparse Blossom implementation of PyMatching [5, 6], is the common reference for both. Neural pre-decoders are one proposal to improve on it: a small convolutional network looks at a window of syndrome rounds, removes the errors it is sure of, and leaves a sparser syndrome to the matching decoder [7, 8]. NVIDIA released two such networks with pretrained weights and evaluation code [8–10]. In earlier work we implemented them as an integer kernel on Alveo U55C cards and measured their throughput on four cards [11].

Those networks were trained, and their published accuracy measured, on circuit noise of one strength at every location. A device is different. On the IBM devices studied here the two qubit gate error of the worst usable coupler is 11 to 130 times that of the median one, the median readout error is 1.1 to 5.6 times the median two qubit gate error, and data qubits idle through microseconds of ancilla readout with their own T_1 and T_2. Whether a network trained on uniform noise keeps its advantage under such noise, and against which matching baseline that advantage should be judged, is the subject of this paper. A decoder that knows each qubit's rates can also weight its matching graph accordingly [12], which raises a second question: how much of any gain comes from the network and how much is available to matching alone.

We answer both questions with device calibration data, without access to the devices. IBM publishes calibration snapshots of its devices with qiskit-ibm-runtime [13]; we export eight of them to CSV files in the layout of IBM's own calibration downloads and map every fault location of NVIDIA's memory circuit to the rate of the qubit or coupler it is assigned to. The pre-decoder then runs unchanged, in floating point and as the integer model that the FPGA kernel computes, and on one U55C card.

The contributions are:

- A noise model that keeps the fault locations of NVIDIA's circuit, so the network sees the structure it was trained on, and sets every location from a calibration CSV: gate errors, readout and preparation errors, and T_1 and T_2 decay over the measured durations of gate, readout and reset layers (Section 3).
- An evaluation over 108 points (eight devices, three placements, three error scales, distances 3 and 5) with paired, shot by shot comparisons against plain matching, matching with device median weights and correlated matching [14, 15] (Section 4).
- The finding that correlated matching, with no network, beats the pre-decoder pipeline at every distance 5 point, that the network loses its advantage on strongly uneven qubits, and that calibrated matching weights are worth up to 1.16 times in logical error on their own.
- Integer inference of the network on one U55C card, bit exact on every run and identical to the CPU integer model in every logical failure wherever both were run, with a direct throughput comparison against a 256 thread CPU.

- A public repository with the calibration files, the experiments and a teaching example that runs codes of distance 1, 2, 3 and 5 from any calibration CSV on a laptop.

2 Related work

Matching and its refinements. Minimum weight perfect matching on the detector graph of a circuit [1, 2] is the standard surface code decoder, and Sparse Blossom made it fast enough to decode a million errors per core second [6]. Union find trades a little accuracy for almost linear time [16] and has been placed on FPGAs [17] and in a real time decoding system [18]. Matching treats X and Z errors independently, so a Y error, or a two qubit gate error that flips both, is paid twice. Correlated matching corrects this by reweighting edges after a first pass [14, 15]; belief matching does so with belief propagation [19]. PyMatching exposes correlated matching through a decoding option, which is the strongest network free baseline used here.

Neural decoders and pre-decoders. Neural decoders for topological codes go back to [20]; pre-decoders that handle local errors in front of a global decoder to [7], and their hardware cost was studied in [21]. AlphaQubit decoded experimental data with a transformer trained on simulated and device data [22]. NVIDIA's Ising pre-decoders [8] are three dimensional convolutional networks with pretrained weights [9] and open evaluation code [10]; their accuracy is reported on uniform circuit noise against plain matching. Our earlier work [11] implemented them as an integer kernel, in the style of integer only inference [23] and FPGA network compilers [24, 25], and measured throughput on four U55C cards. Neither study placed the networks under device calibrated noise or compared them with correlated matching.

Device informed noise and decoder priors. Google's below threshold experiments decoded with priors fitted to device data [3, 4], and [12] showed that optimizing the priors of the matching graph lowers the logical error on hardware. Pauli twirled amplitude and phase damping is the usual way to turn T_1 and T_2 into circuit noise [26], and randomized benchmarking [27] supplies the gate errors that calibration files report. IBM devices use a heavy hexagonal lattice, on which surface codes need flag qubits or a modified layout [28]; we do not embed the code on the lattice (Section 3), but draw its noise from the measured distributions.

3 Methods

3.1 Calibration data

IBM distributes snapshots of device calibrations with the fake backends of qiskit-ibm-runtime [13] (version 0.50.0, Apache License 2.0). We export 8 of them to CSV files in the column layout of IBM's calibration downloads: per qubit T_1, T_2, readout assignment error, the probability of reading 1 after preparing 0, readout length, $\sqrt{X}$ error and gate length, reset length, and per coupler the CZ or ECR error and gate length. The reader matches column headers by keywords, so a CSV downloaded from IBM's dashboard loads unchanged; a qubit with any missing value is marked unusable

Table 1 The eight calibration snapshots: medians over usable qubits and couplers.

Device	Family (snapshot)	Usable qubits	2q gate	2q error (%)	sx error (%)	Readout error (%)	T_1 (μs)	T_2 (μs)
Aachen	Heron r3 (2026-04)	156	CZ	0.15	0.019	0.74	232	255
Kingston	Heron r3 (2026-04)	155	CZ	0.18	0.023	0.95	283	144
Marrakesh	Heron r2 (2025-02)	156	CZ	0.32	0.023	0.95	197	118
Fez	Heron r2 (2025-02)	156	CZ	0.38	0.023	0.76	145	88
Torino	Heron r1 (2025-02)	132	CZ	0.41	0.027	2.29	185	141
Sherbrooke	Eagle r3 (2025-02)	126	ECR	0.75	0.024	1.97	279	169
Brisbane	Eagle r3 (2025-02)	127	ECR	0.77	0.024	2.00	232	150
Kyiv	Eagle r3 (2025-02)	127	ECR	1.15	0.025	1.27	287	118

Table 2 Fault locations of NVIDIA's circuit and their calibrated channels. r is an error per gate from randomized benchmarking; idle channels use the duration of the layer in which the qubit waits.

Location	Calibrated channel
Two qubit gate	two qubit depolarizing with $5r_{2q}/4$ on the pair; one qubit depolarizing with $3r_{sx}$ on the target (the two Hadamards that turn CZ or ECR into CNOT)
Idle during gates	Pauli twirled T_1 and T_2 decay over the longest coupler of the layer plus two $\sqrt{X}$ gates
Idle during readout	the same, over the longest ancilla readout
Idle during reset	the same, over the longest ancilla reset
Measurement	bit flip with the readout assignment error
Preparation	bit flip with half of the probability of reading 1 after preparing 0

and never placed. Table 1 lists the devices: two Heron r3 snapshots from April 2026, and Heron r2, Heron r1 and Eagle r3 snapshots from February 2025.

3.2 Calibrated noise on NVIDIA's memory circuit

The experiment is a rotated surface code memory of distance d over d rounds in the X or Z basis, built by NVIDIA's `MemoryCircuit` with boundary detectors [10]: the circuit on which the pre-decoder was trained and evaluated. NVIDIA's noise model places noise in every round except the last, which is perfect, and reads the data qubits without noise. We generate the circuit with distinct placeholder probabilities, one per class of fault location, and replace every placeholder by a channel computed from the calibration of the device qubit or coupler assigned to that location (Table 2). Detector and observable definitions are untouched, so the network input, the kernel input and the detector order seen by PyMatching are those of NVIDIA's pipeline.

For an idle of duration t the Pauli twirl of amplitude and phase damping [26] gives

$$p_x = p_y = \frac{1 - e^{-t/T_1}}{4}, \qquad p_z = \frac{1 - e^{-t/T_2}}{2} - p_x, \tag{1}$$

with T_2 clipped to $2T_1$. Average gate errors r are converted to depolarizing probabilities with $3r/2$ for one qubit and $5r/4$ for two qubits. Measurement and preparation flips are told apart by the next operation on the qubit: a flip followed by a measurement of that qubit is a readout error, any other is a preparation error. The preparation error takes half of IBM's "Prob meas1 prep0" because that quantity contains both the preparation and the readout of a prepared 0. An error scale s multiplies every probability and every decay rate ($t \to st$ in Eq. 1), so $s = 0.5$ models a device with half of every error.

Placement. IBM devices are heavy hexagonal and the rotated surface code needs a square grid. We do not embed the code in the device graph, which would require flag qubits or SWAPs [28]. Instead, each code qubit gets a distinct device qubit and each code coupler a distinct device coupler, in one of three modes: *median* (every location gets the device median, a homogeneous circuit), *best* (the lowest error usable qubits and couplers) and *screened* (a seeded random draw after removing the worst 10% of every metric). The noise is therefore drawn from each device's measured distribution, not a model of running the code on that chip.

Experimental points. Every device is run in the median and best modes at $s = 1$, 0.5 and 0.25 and $d = 3$ and 5; the two Heron r3 devices are also run with three screened draws at $s = 0.5$. Each point is sampled with Stim [29], 200,000 shots per basis, with fixed seeds that make every decoder see the same shots. The X and Z experiments are pooled, giving 108 points of 400,000 shots.

3.3 Decoders

All matching decoders use PyMatching 2.4 [5, 6] on the decomposed detector error model of a circuit:

PM plain matching on the calibrated circuit, so every edge has the weight of its own qubits' rates;
PM median plain matching on the median circuit of the same device, which knows only the averages;
PM-corr matching with correlations enabled, on the calibrated circuit;
Pre-decoder + PM NVIDIA's Fast network (four 3D convolutions of width 128, receptive field 9, trained at $d = 9$ on uniform noise [9]) in float32 or as the INT8 integer model, followed by PM on the residual syndrome, through NVIDIA's evaluation module;
Pre-decoder + PM-corr the float network followed by PM-corr.

The integer model uses INT8 weights with one scale per output channel, INT8 activations with per layer scales taken at the 99.99th percentile of 1,024 windows sampled from the same circuit with a different seed, and the GELU as a lookup table; it is the model the FPGA kernel computes [11]. A logical failure is a shot whose predicted observable flip differs from the true one. For two decoders a and b on the same shots we report the ratio of failures and the two sided exact sign test on the shots where exactly one of them fails; a difference is called significant at $p < 0.05$. The logical error per round is $\epsilon = \frac{1}{2}\left(1 - (1 - 2P)^{1/r}\right)$ for a failure probability P over r rounds.

3.4 One U55C card

The card runs the bitstream of the earlier four card study [11] (SHA256 prefix d399f8d0) on one Alveo U55C [30]: two instances of an integer kernel whose window size, up to $13 \times 13 \times 13$, and weights are launch arguments. A C++ host on XRT [31] streams the test windows of a point to both instances and reads back one decision bit per site. On the first 4,096 windows of every run, both instances are checked bit exact against the integer model. The decisions of all 200,000 windows are then passed, as fixed network outputs, through the same evaluation module and PM, and the logical failures are compared shot by shot with the CPU integer model. Throughput is measured with both instances busy and two launches in flight, 64 windows per launch, over three repetitions of about ten seconds; latency is the mean time of a one window launch with one launch in flight, PCIe transfers and runtime included. For comparison, the float network is timed in PyTorch on the AMD EPYC 7742 64-Core host with 256 threads, network forward pass only, in batches of 4,096 windows.

3.5 Teaching example

Distances 1 and 2 cannot correct errors and are not part of the evaluation. The repository includes a CPU example that runs them, together with $d = 3$ and 5, from any calibration CSV with the device medians: $d = 1$ is one unencoded qubit kept for the duration of the rounds, $d = 2$ discards every shot with a detection event, and $d = 3$ and 5 are decoded by PM. With the Kingston medians over five rounds it gives logical errors of 5.37% for the bare qubit, 0.181% for the kept shots of $d = 2$ (with 51% of shots discarded), 3.33% for $d = 3$ and 1.526% for $d = 5$ at $s = 1$, and 0.028% for $d = 5$ at $s = 0.25$. It uses Stim's generated circuits with a simpler noise model and is not compared with the results below.

4 Results

4.1 Logical error per round

Figure 1 shows the logical error per round of the homogeneous (median) circuits. At the calibrated rates ($s = 1$) the error per round falls from $d = 3$ to $d = 5$ on 3 of the 8 devices with PM and on 6 with PM-corr; at $s = 0.5$ and $s = 0.25$ it falls on all of them with every decoder. The two Heron r3 devices are the lowest at every scale, the three Eagle r3 devices the highest. These are properties of the calibrations under our noise model, not hardware results, but they set the regime: the snapshots place most devices near threshold for this circuit, and the scaled points place them below it.

4.2 The pre-decoder against plain and correlated matching

Table 3 counts the significant wins and losses of every comparison over all points, and Fig. 2 shows every failure ratio against the logical error of PM.

Against plain matching the pre-decoder helps, as NVIDIA reports for uniform noise [8]: it is significantly better at 35 of 54 points at $d = 3$ and 44 of 54 at $d = 5$, with failure ratios down to 0.85 and 0.64. The gain is largest on Aachen, the lowest

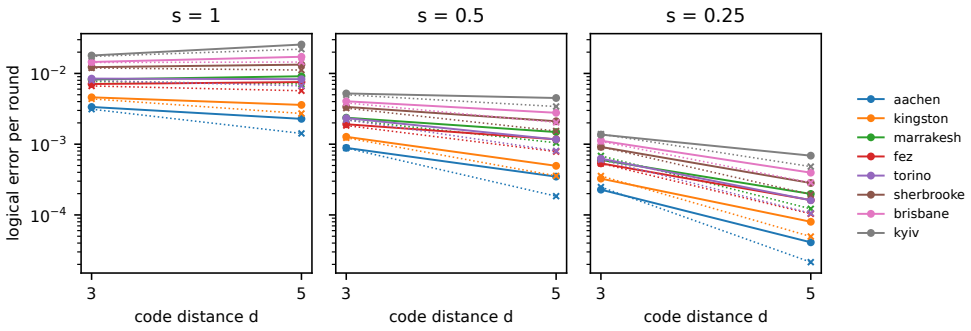


Fig. 1 Logical error per round of the median circuits, X and Z pooled. Solid lines: PM; dotted lines: PM-corr. Panels: error scale s.

Table 3 Paired comparisons over all placements and scales: points where the first decoder fails significantly less (Better) or more (Worse) often than the second (exact sign test, $p < 0.05$), and the range of failure ratios. The median weight comparison uses only the best and screened placements, where the two graphs differ.

Comparison	d	Points	Better	Worse	Failure ratio
INT8 pre-decoder + PM vs PM	3	54	35	4	0.85 to 1.17
INT8 pre-decoder + PM vs PM	5	54	44	4	0.64 to 1.06
INT8 pre-decoder + PM vs PM-corr	3	54	21	12	0.73 to 1.20
INT8 pre-decoder + PM vs PM-corr	5	54	0	51	1.07 to 1.61
float pre-decoder + PM vs PM-corr	3	54	17	19	0.76 to 1.24
float pre-decoder + PM vs PM-corr	5	54	0	49	0.99 to 1.60
float pre-decoder + PM-corr vs PM-corr	3	54	20	15	0.76 to 1.24
float pre-decoder + PM-corr vs PM-corr	5	54	0	49	0.99 to 1.61
PM-corr vs PM	3	54	25	8	0.90 to 1.29
PM-corr vs PM	5	54	54	0	0.50 to 0.87
PM, median weights vs calibrated	3	30	0	19	0.99 to 1.16
PM, median weights vs calibrated	5	30	0	20	0.97 to 1.15

error device, and smallest on Torino and the Eagle devices, where the ratio approaches or exceeds 1 at the calibrated rates.

Against correlated matching the result reverses at $d = 5$. PM-corr alone fails less often than the pre-decoder pipeline at every one of the 54 points, significantly at 51, with ratios from 1.07 to 1.61. On Aachen at the calibrated rates, for example, PM fails 4,520 times in 400,000 shots, the pre-decoder pipeline 3,347 and PM-corr 2,823. PM-corr itself beats PM at every $d = 5$ point (ratios 0.50 to 0.87), so most of what the network gains over plain matching is the same class of errors that correlated matching already handles: two qubit gate faults and Y errors that flip both an X and a Z detector. The float network gives the same picture (Table 3), so this is not an effect of quantization. Placing the network in front of PM-corr does not help either (0 better, 49 worse at $d = 5$; on Aachen 3,306 failures).

At $d = 3$ the picture is mixed. PM-corr is worse than PM at 13 of the 16 points at $s = 0.25$, and there the pre-decoder beats PM-corr significantly at 13 of them; overall

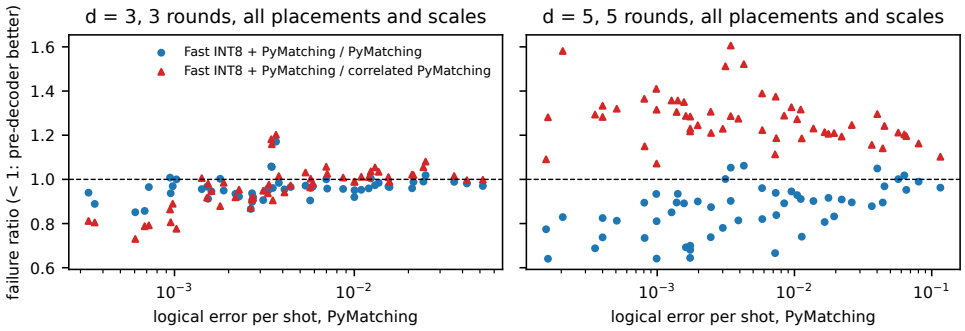


Fig. 2 Failures of the INT8 pre-decoder followed by PM, divided by the failures of PM (circles) and of PM-corr (triangles), at every point; X and Z pooled, 400,000 shots per point.

at $d = 3$ the pre-decoder is better than PM-corr at 21 points and worse at 12. In this regime, small codes at low error, the network is the better addition to plain matching.

4.3 Uneven qubits

The best placements, whose qubits are all good but differ, behave like the median ones (mean $d = 5$ ratio against PM 0.83, against 0.88 for median). The screened placements draw from most of a device's spread. On Aachen they still behave like the median ones, but on Kingston the pre-decoder makes plain matching significantly worse at 5 of 6 points, with ratios up to 1.17; these are most of the significant losses in Table 3, the others being Torino and Sherbrooke at the calibrated rates. A network trained on uniform noise removes errors as if every location were equally likely. When the rates of the locations differ widely, some of those removals are wrong, while PM, which weights each edge by its own rate, is not misled in the same way. We read this as a limit of the training distribution rather than of pre-decoding, which training on calibrated noise could test.

4.4 Calibrated matching weights

On the best and screened placements the calibrated circuit and the median circuit differ, and PM can be given either graph. Figure 3 shows the failure ratio of the median graph to the calibrated one. Median weights are never significantly better, and are significantly worse at 19 of 30 points at $d = 3$ and 20 of 30 at $d = 5$ (Table 3), by up to 1.16 times. This gain needs no network and no accelerator, only the calibration file, and it is of the same size as the network's gain over PM at $d = 3$.

4.5 One U55C card

Table 4 lists the 18 card runs. Both kernel instances were bit exact against the integer model in all runs, and in all 14 runs that have a CPU integer counterpart the set of shots on which the card pipeline fails is identical to that of the CPU integer model. The $d = 7$ runs on Kingston, outside the CPU campaign, show no gain over PM at that distance: the card pipeline fails more often than PM in three of the four runs.

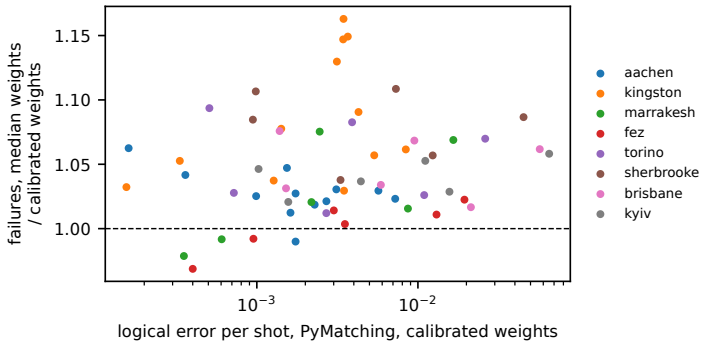


Fig. 3 Failures of PM with the device median graph divided by failures of PM with the calibrated graph, best and screened placements, against the logical error of the calibrated PM.

Table 4 Card runs: logical failures of PM and of the card's INT8 decisions followed by PM on the same 200,000 shots, and whether the failing shots equal those of the CPU integer model (n/a: no CPU integer run at that point).

Device	Scale	d	Basis	Shots	PM	Card + PM	Same as CPU
Kingston	1	3	X	200,000	3,789	3,659	yes
Kingston	1	3	Z	200,000	1,677	1,721	yes
Kingston	1	5	X	200,000	5,257	4,847	yes
Kingston	1	5	Z	200,000	1,820	1,637	yes
Kingston	1	7	X	200,000	5,286	5,517	n/a
Kingston	1	7	Z	200,000	1,355	1,409	n/a
Kingston	0.5	3	X	200,000	1,050	1,026	yes
Kingston	0.5	3	Z	200,000	472	472	yes
Kingston	0.5	5	X	200,000	741	658	yes
Kingston	0.5	5	Z	200,000	246	205	yes
Kingston	0.5	7	X	200,000	432	483	n/a
Kingston	0.5	7	Z	200,000	97	88	n/a
Aachen	1	3	X	200,000	1,898	1,757	yes
Aachen	1	3	Z	200,000	2,105	1,925	yes
Aachen	1	5	X	200,000	2,133	1,562	yes
Aachen	1	5	Z	200,000	2,387	1,785	yes
Sherbrooke	0.5	3	X	200,000	2,284	2,164	yes
Sherbrooke	0.5	5	X	200,000	2,407	2,245	yes

Table 5 gives throughput and latency. One card decodes 4,201 windows per second at $d = 3$ and 2,330 at $d = 5$; a single window takes 1.05 and 1.50 ms through PCIe. The float network on 256 CPU threads is faster at every distance measured, by 5.1 times at $d = 3$ and 1.6 times at $d = 9$, the gap narrowing as the windows grow. At milliseconds per window, neither is close to the microsecond syndrome cycle of superconducting devices.

Table 5 Throughput of one card (both kernel instances) and latency of one window, against the float network on the CPU host (forward pass only, batches of 4,096).

d	Card (windows/s)	One window (ms)	CPU (windows/s)	CPU / card
3	4,201	1.05	21,289	5.1
5	2,330	1.50	5,595	2.4
7	1,171	2.34	2,082	1.8
9	622	3.84	1,016	1.6
11	360	6.21	n/a	n/a
13	224	9.56	n/a	n/a

5 Discussion

Which baseline. A pre-decoder is usually judged against the matching decoder it feeds. Our results show why that is not enough under realistic noise: most of what the network gains over plain matching at $d = 5$ is recovered, and more, by correlated matching, a decoding option of the same library. The network's advantage at $d = 3$ and low error is real but small, and it sits where correlated matching is at its weakest. Any claim of a pre-decoder gain under device noise should therefore name the strongest network free baseline at the same point. Belief matching [19] and other correlated decoders were not run and might move the bar further.

What calibration buys. The calibration file helps twice. As a noise source it exposes the network to structure it was not trained on, and on strongly uneven placements the network's uniform prior costs logical fidelity. As a set of matching weights it improves plain matching by up to 1.16 times with no network at all, a gain of the same size as the network's own at $d = 3$. The obvious next step, training or fine tuning the network on calibrated noise, is outside this study; the released noise model produces exactly the training circuits it would need.

What the card shows. The card reproduces the integer model exactly, shot for shot in logical outcome, so the FPGA adds no error beyond quantization. Quantization itself is not free under calibrated noise: the integer pipeline fails 0.92 to 1.17 times as often as the float one, significantly better at 28 and worse at 26 of 108 points. The integer scales are fitted to each circuit's own windows; a deployed integer pre-decoder would need the same per device calibration. As an accelerator, one card is slower than the 256 thread host for this small network, and at hundreds of microseconds per round it is far from a microsecond syndrome cycle. The value of the card in this study is as a verified integer implementation, not as a speedup.

6 Limitations

No circuit was run on IBM hardware. The calibrations set the rates of a simulated circuit; leakage, crosstalk, measurement induced dephasing beyond T_2, drift and correlated errors are not modelled, and gate errors from randomized benchmarking are treated as depolarizing. The code is not embedded in the heavy hexagonal lattice, so the results describe noise drawn from each device's distribution, not that device running the code. The snapshots are single points in time. NVIDIA's network was used as

published, trained at $d = 9$ on uniform noise. Throughput and latency are measured through PCIe with runtime overheads, not in a decoding loop.

7 Conclusions

We evaluated a published neural pre-decoder on surface code circuits whose noise comes from the calibration files of eight IBM devices, over 108 points and 43,200,000 shots, and ran its integer form on one Alveo U55C. Against plain matching the pre-decoder helps at most points, as reported for uniform noise. Against correlated matching, which needs no network, it loses at every distance 5 point, and putting it in front of correlated matching does not help; it keeps an edge only for small codes at low error. On strongly uneven qubits the network trained on uniform noise makes plain matching worse, while matching weights from the calibration file improve it. The card computes the integer network exactly, with logical outcomes identical to the CPU model wherever both were run, but more slowly than a large CPU. A pre-decoder for real devices will need to be trained on device like noise and judged against correlated matching; the released calibration files, noise model and experiments are a basis for both.

Acknowledgments. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India. The research was conducted at the C-DAC Noida Centre under the supervision of Mr. Abhishek Tiwari, Scientist F, Embedded Systems Division, C-DAC Noida. We thank NVIDIA for releasing the Ising pre-decoders with their evaluation code and pretrained weights, IBM for publishing device calibration snapshots with qiskit-ibm-runtime, and the developers of Stim and PyMatching, on which the evaluation of this work depends.

Declarations

Funding. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India.

Conflict of interest. The author declares no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The calibration files, the per shot failure records of every decoder at every experimental point, the card measurement records and the scripts that generate every table and figure from them are available from the corresponding author on reasonable request. The calibration snapshots are distributed with qiskit-ibm-runtime under the Apache License 2.0 [13]; the pretrained network is available from NVIDIA under the NVIDIA Open Model License [9].

Author contribution. N. Ali designed the calibration driven noise model and the experiments, implemented the evaluation harness and the card runtime, carried out the measurements and wrote the manuscript.

References

[1] Dennis, E., Kitaev, A., Landahl, A., Preskill, J.: Topological quantum memory. Journal of Mathematical Physics **43**(9), 4452–4505 (2002) https://doi.org/10.1063/1.1499754

[2] Fowler, A.G., Mariantoni, M., Martinis, J.M., Cleland, A.N.: Surface codes: Towards practical large-scale quantum computation. Physical Review A **86**(3), 032324 (2012) https://doi.org/10.1103/physreva.86.032324

[3] Google Quantum AI: Suppressing quantum errors by scaling a surface code logical qubit. Nature **614**(7949), 676–681 (2023) https://doi.org/10.1038/s41586-022-05434-1

[4] Google Quantum AI and Collaborators: Quantum error correction below the surface code threshold. Nature **638**(8052), 920–926 (2025) https://doi.org/10.1038/s41586-024-08449-y

[5] Higgott, O.: Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching. ACM Transactions on Quantum Computing **3**(3), 1–16 (2022) https://doi.org/10.1145/3505637

[6] Higgott, O., Gidney, C.: Sparse blossom: correcting a million errors per core second with minimum-weight matching. Quantum **9**, 1600 (2025) https://doi.org/10.22331/q-2025-01-20-1600

[7] Chamberland, C., Ronagh, P.: Deep neural decoders for near term fault-tolerant experiments. Quantum Science and Technology **3**(4), 044002 (2018) https://doi.org/10.1088/2058-9565/aad1f7

[8] Chamberland, C., Olle, J., Li, M., Thornton, S., Baratta, I.: Fast and accurate ai-based pre-decoders for surface codes. arXiv preprint arXiv:2604.12841 (2026) arXiv:2604.12841

[9] NVIDIA: Ising-Decoder-SurfaceCode-1-Fast, model card, checkpoint v1.0.77. https://huggingface.co/nvidia/Ising-Decoder-SurfaceCode-1-Fast (2026)

[10] NVIDIA: Ising-Decoding: open source training and inference code of the Ising pre-decoders, commit 33acb15. https://github.com/NVIDIA/Ising-Decoding (2026)

[11] Ali, N.: u55c-neural-predecoder-nodes: integer neural pre-decoding for the surface code on four Alveo U55C cards. https://github.com/nasir26/u55c-neural-predecoder-nodes (2026)

[12] Sivak, V., Newman, M., Klimov, P.: Optimization of decoder priors for accurate quantum error correction. Physical Review Letters **133**(15), 150603 (2024) https://doi.org/10.1103/physrevlett.133.150603

[13] IBM Quantum: qiskit-ibm-runtime, version 0.50.0, fake provider backend snapshots. https://github.com/Qiskit/qiskit-ibm-runtime. Apache License 2.0 (2026)

[14] Fowler, A.G.: Optimal complexity correction of correlated errors in the surface code. arXiv preprint arXiv:1310.0863 (2013) arXiv:1310.0863

[15] Paler, A., Fowler, A.G.: Pipelined correlated minimum weight perfect matching of the surface code. Quantum **7**, 1205 (2023) https://doi.org/10.22331/q-2023-12-12-1205

[16] Delfosse, N., Nickerson, N.H.: Almost-linear time decoding algorithm for topological codes. Quantum **5**, 595 (2021) https://doi.org/10.22331/q-2021-12-02-595

[17] Liyanage, N., Wu, Y., Deters, A., Zhong, L.: Scalable quantum error correction for surface codes using fpga. In: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 916–927 (2023). https://doi.org/10.1109/qce57702.2023.00106

[18] Barber, B., Barnes, K.M., Bialas, T., Buğdaycı, O., Campbell, E.T., Gillespie, N.I., Johar, K., Rajan, R., Richardson, A.W., Skoric, L., Topal, C., Turner, M.L., Ziad, A.B.: A real-time, scalable, fast and resource-efficient decoder for a quantum computer. Nature Electronics **8**(1), 84–91 (2025) https://doi.org/10.1038/s41928-024-01319-5

[19] Higgott, O., Bohdanowicz, T.C., Kubica, A., Flammia, S.T., Campbell, E.T.: Improved decoding of circuit noise and fragile boundaries of tailored surface codes. Physical Review X **13**(3), 031007 (2023) https://doi.org/10.1103/physrevx.13.031007

[20] Torlai, G., Melko, R.G.: Neural decoder for topological codes. Physical Review Letters **119**(3), 030501 (2017) https://doi.org/10.1103/physrevlett.119.030501

[21] Overwater, R.W.J., Babaie, M., Sebastiano, F.: Neural-network decoders for quantum error correction using surface codes: A space exploration of the hardware cost-performance tradeoffs. IEEE Transactions on Quantum Engineering **3**, 1–19 (2022) https://doi.org/10.1109/tqe.2022.3174017

[22] Bausch, J., Senior, A.W., Heras, F.J.H., Edlich, T., Davies, A., Newman, M., Jones, C., Satzinger, K., Niu, M.Y., Blackwell, S., Holland, G., Kafri, D., Atalaya, J., Gidney, C., Hassabis, D., Boixo, S., Neven, H., Kohli, P.: Learning high-accuracy error decoding for quantum processors. Nature **635**(8040), 834–840 (2024) https://doi.org/10.1038/s41586-024-08148-8

[23] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2704–2713 (2018). https://doi.org/10.1109/

cvpr.2018.00286

[24] Duarte, J., Han, S., Harris, P., Jindariani, S., Kreinar, E., Kreis, B., Ngadiuba, J., Pierini, M., Rivera, R., Tran, N., Wu, Z.: Fast inference of deep neural networks in fpgas for particle physics. Journal of Instrumentation **13**(07), 07027–07027 (2018) https://doi.org/10.1088/1748-0221/13/07/p07027

[25] Umuroglu, Y., Fraser, N.J., Gambardella, G., Blott, M., Leong, P., Jahre, M., Vissers, K.: Finn. In: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 65–74 (2017). https://doi.org/10.1145/3020078.3021744

[26] Ghosh, J., Fowler, A.G., Geller, M.R.: Surface code with decoherence: An analysis of three superconducting architectures. Physical Review A **86**(6), 062318 (2012) https://doi.org/10.1103/physreva.86.062318

[27] Magesan, E., Gambetta, J.M., Emerson, J.: Scalable and robust randomized benchmarking of quantum processes. Physical Review Letters **106**(18), 180504 (2011) https://doi.org/10.1103/physrevlett.106.180504

[28] Chamberland, C., Zhu, G., Yoder, T.J., Hertzberg, J.B., Cross, A.W.: Topological and subsystem codes on low-degree graphs with flag qubits. Physical Review X **10**(1), 011022 (2020) https://doi.org/10.1103/physrevx.10.011022

[29] Gidney, C.: Stim: a fast stabilizer circuit simulator. Quantum **5**, 497 (2021) https://doi.org/10.22331/q-2021-07-06-497

[30] Advanced Micro Devices: Alveo U55C Data Center Accelerator Card Data Sheet (DS978). https://docs.amd.com/r/en-US/ds978-u55c (2023)

[31] Advanced Micro Devices: Xilinx Runtime (XRT) Documentation. https://xilinx.github.io/XRT/ (2023)